

# Software Supply Chain

What we found, what it would cost, and what we would do

11 August 2026 · Internal

# Before anything else

---

## Nothing has happened to us.

- We went looking for evidence of an attack
- We did not find any
- This is about the doors, not a burglary

### SPEAKING NOTES

Say this first and say it plainly. Any room handed a security deck assumes there has been a breach. If you do not kill that assumption in the first thirty seconds, everything else in this deck gets heard as damage control.

We checked what our software is built from, what our automated systems have been doing, and what our laptops and servers have been running. Nothing showed a break-in.

# How software actually gets built

---

We do not write it all ourselves.

- Products are **assembled** from thousands of free components
- Published by strangers, downloaded automatically
- Like buying screws and motors instead of making them

This is normal. It is how the whole industry works.

## SPEAKING NOTES

The audience needs this before anything else lands. Nobody sits down and writes a modern application from nothing — they assemble it from parts. Our automated build systems pull those parts down constantly, with no person watching.

Do not apologize for this. It is not a bad practice. It is the practice.

## ...which is why attackers now aim there

---

1. Take over the account of someone who publishes a popular component
2. Publish a new version with hostile code inside
3. Every company that builds software that week picks it up

**Nobody clicked anything. Nobody made a mistake.**

### SPEAKING NOTES

The component was trusted last week, so it was trusted this week. That is the whole trick.

This has been happening across the industry at scale through 2026. The question this briefing answers is not “did we get hit” — we did not — it is “how well would we hold up.”

# Five problems, one chain

---

- We **run** what suppliers send us, unread
- We hand our build systems **every key we own**
- Nine of our processes call an **address anyone can claim**
- We track **moving targets** instead of fixed ones
- Keys **never expire**, and there is no one place to say no

## SPEAKING NOTES

Emphasize: these are five separate problems, and they are only serious *together*. They line up into a single chain. Each of the next few slides is one link.

# 1. We hand over the whole keyring

---

A contractor comes to fix the supply closet. We give them the master key to **every door in the building**.

|                                            |              |
|--------------------------------------------|--------------|
| Places this happens                        | <b>1,924</b> |
| Through one single component               | <b>1,904</b> |
| Projects reached by shared building blocks | <b>1,823</b> |
| ...that deploy to production               | <b>37</b>    |

## SPEAKING NOTES

The concentration is the good news. 1,904 of 1,924 go through one shared component, so this is a handful of things to fix, not thousands.

If someone asks why it was built this way: it was convenient, and it worked. Nobody did anything reckless. It just was never revisited.

## 2. We run supplier code on sight

---

Components arrive with a note attached: *“before you use me, run this.”*

**We run it. Automatically. Before anyone reads it.**

- Only **6** of our projects refuse
- **94** run whatever arrives

### SPEAKING NOTES

This is the front door of the attack — where these campaigns begin, every time.

There is a single setting that says “download the component, but do not run its instructions.” Almost nothing of ours sets it. That is the cheapest fix in the entire deck.

### 3. Nine open doors

---

Nine automated processes are asking for instructions from an address where **nothing lives any more.**

- The address is **vacant, not sealed**
- Anyone inside the company can move in
- Whatever they supply gets handed the keyring

**No outside attacker needed.**

#### SPEAKING NOTES

Right now these nine are simply broken. That is the mild version.

The serious version: the addresses are not secret — anyone can read our own project files and see which ones we are calling. Somebody inside with ordinary access could claim one.

This is the sharpest thing we found, and it is hours of work to fix.

## 4. We track moving targets

---

We ask for “the current version” rather than “exactly the one we checked.”

**Roughly 1 in 10** of our references name a fixed version. The other nine can change under us overnight.

### SPEAKING NOTES

No approval, no change request, no notification. What runs tomorrow can differ from what ran today.

Worth noting this is also a reliability problem, not only a security one — see the benefits slide.

# What happens if we do not fix it

---

- **A second of access becomes every credential we own**
- Cannot prove what was taken → must assume **everything**
- Rotate every key, redeploy, under pressure, work stops
- Hard to answer *“what did they take, and where did they go?”*

There is **no small version** of this incident.

## SPEAKING NOTES

Be concrete rather than dramatic. Nobody is claiming the company falls over.

The expensive part is the clean-up, and it is triggered by an event that might last seconds. That is the asymmetry worth funding against.

# Worth doing even if nobody attacks us

---

- **Nine broken automations** get repaired
- Builds **stop changing underneath us** — fewer mystery failures
- A real **list of what our software is made of** — what auditors ask for
- **Less exposure to someone else's outage**
- Incidents get **shorter**, because keys expire on their own

## SPEAKING NOTES

This is the slide that gets the work funded. Several of these fixes pay for themselves with no attacker involved at all.

The inventory point lands especially well with anyone who has been asked by a customer or an auditor what our software contains. Today that question takes a project to answer.

# Now — weeks, not quarters, nothing to buy

---

- A. Close the nine open doors** — hours Repoint them, and lock the names so nobody can claim them
- B. Stop running supplier code on sight** — one line per project 94 projects, cheapest protection here
- C. Stop handing over the whole keyring** — a few weeks A small number of shared components, not thousands of projects

## SPEAKING NOTES

Order matters. A first — it is the sharpest risk and the least effort.

On A: both halves. Repointing without locking the name leaves the door shut but unlocked.

On C: it sounds enormous and it is not, because the exposure is concentrated in a few shared components.

# Next — a quarter each

---

**D. Badges instead of keys** Short-lived passes issued per job, expiring in minutes. Nothing durable left to steal.

**E. One front door for outside software** One place to block a bad component everywhere at once, and a short waiting period before new versions can be used.

## SPEAKING NOTES

D is what makes C permanent rather than a setting somebody can quietly undo later. We already own this capability and are barely using it.

Neither of these is a purchase. Both are configuration of things already licensed.

# We already own the front door

---

Almost nothing goes through it. **The mechanism is proven inside our own walls** — it was simply never pointed at this category of software.

## Through our internal supply point

|                                |     |
|--------------------------------|-----|
| Microsoft-ecosystem components | 523 |
|--------------------------------|-----|

|                                 |   |
|---------------------------------|---|
| The components in question here | 0 |
|---------------------------------|---|

### SPEAKING NOTES

This is the strongest slide in the deck for anyone worried about cost. We are not proposing to build something. It exists, it is connected, and one category of software already flows through it successfully.

# Why a waiting period works

The catalog cleans itself up — but only after a few days, and **our builds are fast enough to catch them in that gap.**

|                                                 |              |
|-------------------------------------------------|--------------|
| <b>Bad versions published in this campaign</b>  | <b>2,208</b> |
| Already pulled from the catalog when we checked | <b>2,097</b> |
| Still out there                                 | <b>111</b>   |

**SPEAKING NOTES**

Around 95% withdrawn. Make a version sit unused for a few days and most of these are gone before we could ever have touched them.  
The remaining 111 are exactly why we need the block list as well as the delay. Do not oversell the waiting period as a complete answer.

# What we are not claiming

---

- We did **not** check everything — **193** projects unread
- **Not checked is not the same as clean**
- Personal laptops are largely **outside** this
- One internal supply point we **could not read**

**“No evidence of a break-in” is not “no break-in.”**

## SPEAKING NOTES

Include this slide. Do not cut it for time.

A briefing that overstates its confidence is worth less than one that does not, and this audience has no way to catch us at it. Saying where we could not look is what makes the rest credible.

Most real victims of these campaigns were hit on a developer's own laptop, not a build server. Our visibility there is thinner.

# In one minute

---

We build our products partly from free components downloaded automatically from the internet. Attackers have started poisoning those at the source, and it is working across the industry.

**We checked. It has not reached us.**

If it did, three things would turn a small event into a large one — and all three cost **weeks, not quarters**, with **nothing to purchase**.

## SPEAKING NOTES

If you get five minutes instead of thirty, this is the slide. Open on it, and take questions.

The three: we run supplier code on sight, we hand over every key instead of one, and nine processes call an address anyone inside could claim.